Non-interactive Butterfly Counting in Bipartite Graphs Based on Local Differential Privacy

Jiayu Li[†‡§], Bin Kuang[†‡§], Shiqi Zhou[†‡§], Yuxiang Wang[*†§], Jin Cao[‖], Ben Niu[†§],
[†]Institute of Information Engineering, Chinese Academy of Sciences, P.R. China
[‡]School of Cyber Security, University of Chinese Academy of Sciences, P.R. China
[§]State Key Laboratory of Cyberspace Security Defense, P.R. China
[‖]School of Cyber Engineering, Xidian University, P.R. China
{lijiayu, kuangbin, zhoushiqi, wangyuxiang, niuben}@iie.ac.cn, jcao@xidian.edu.cn
[*]Corresponding author

***Abstract*—Counting butterflies in bipartite graphs is a fundamental task that has gained increasing attention. Estimating butterfly counts with edge local differential privacy for privacy protection is meaningful. Existing methods struggle to balance estimation accuracy, running efficiency, user synchronization and communication cost. We propose 2-path-based butterfly counting (PBC), which accurately and efficiently counts the number of butterflies in bipartite graphs in a non-interactive manner. PBC enhances running efficiency by transforming the butterfly count into a function of the 2-path count, and improves estimation accuracy with an unbiased estimation of the butterfly count. Additionally, PBC includes privacy budget allocation that minimizes the variance of the 2-path count. We evaluate PBC on four real-world datasets, demonstrating it's superior estimation accuracy and running efficiency compared to existing methods.**

I. INTRODUCTION

Graph data can be used to describe networks such as social and voting networks. Bipartite graph is a special kind of graph in which nodes are divided into two categories, with edges existing only between nodes of different categories. In real life, networks such as election voting and movie ratings can be represented as bipartite graphs. As shown in Fig. 1, the upper-layer nodes can represent candidates, and the lower-layer nodes can represent voters, edge (U_1, V_1) exists when V_1 votes for U_1. The smallest closed unit in a bipartite graph is called a butterfly. As shown in Fig. 1, we refer to a shape like $U_1V_2U_2$ as a 2-path, and a shape formed by two 2-paths like $U_2V_3U_3V_5$ as a butterfly. Butterfly counting is fundamental for link prediction, recommendation systems, and other tasks in bipartite graphs [1], [2]. However, the edges in a bipartite graph may involve users' privacy, privacy-preserving butterfly counting is meaningful.

Compared to differential privacy (DP) [3], local differential privacy (LDP) [4], [5] provides quantified privacy protection without a trusted third party. LDP is widely used in privacy-preserving graph analysis [1], [2], [6]–[13]. There are already privacy-preserving subgraph counting works utilizing LDP. A study in [7] demonstrates the importance of node degrees and neighbor lists for LDP-based graph analysis. Researchers introduce an additional interaction [6], [9] to achieve high accuracy in triangle counting. However, this interaction requires user synchronization, so [14] proposes non-interactive triangle and 4-cycle counting methods under shuffle DP [15], a 4-cycle is exactly the butterfly in bipartite graphs. Researchers also explore butterfly counting in bipartite graphs [1], [2]. The study in [2] designs non-interactive DBE, which suffers high time complexity. The butterfly counting method proposed by [1] involves complex interactions that require user synchronization. Overall, existing butterfly counting methods cannot balance estimation accuracy, running efficiency, communication cost and user synchronization.

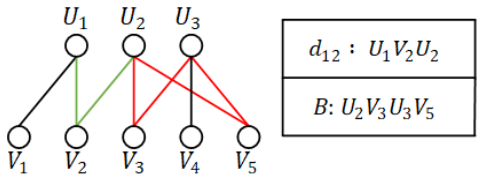


Fig. 1: An example of bipartite graph, $U_1V_2U_2$ forms a 2-path, $U_2V_3U_3V_5$ forms a butterfly.

To address these issues, we propose 2-path-based butterfly counting (PBC), providing edge-LDP [8]. PBC can achieve high estimation accuracy and running efficiency in butterfly counting in a non-interactive manner, requiring low communication cost and no user synchronization. First, leveraging the characteristics of bipartite graphs, server in PBC only collects noisy degrees and neighbor lists from one category of nodes. With the collected data, PBC transforms butterfly count into a function of 2-path count, because the computation of 2-path count is more efficient. Then, through variance analysis of the 2-path counts, we derive an unbiased estimation for butterfly counting, achieving high estimation accuracy. Besides, we allocate privacy budget between degrees and neighbor lists with the variance analysis and limited prior knowledge. Overall, our contributions are as follows:

- We design PBC, a non-interactive butterfly counting method that fully leverages the characteristics of bipartite graphs. PBC transforms the butterfly count into a func-

This work is supported by the National Key R&D Program of China (2022YFB3103404), the National Natural Science Foundation of China (62332018), the Major Programs of the National Social Science Foundation of China(22&ZD147).

ICC 2025 - IEEE International Conference on Communications | 979-8-3315-0521-9/25/$31.00 ©2025 IEEE | DOI: 10.1109/ICC52391.2025.11161705

tion of 2-path count, balancing estimation accuracy and running efficiency.
- Based on variance analysis of 2-paths count, PBC achieves unbiased estimation of butterfly counting. PBC also utilizes variance analysis and limited prior knowledge to allocate privacy budget between the two rounds of data collection, thereby enhancing estimation accuracy.
- We systematically evaluate PBC on four real-world datasets. PBC can achieve a reduction of relative error or running time by more than an order of magnitude compared to state-of-the-art methods. We also evaluate the efficacy of our privacy budget allocation method.

Roadmap. Section II provides preliminary for understanding our research and introduces related work. Section III presents how PBC works, involving unbiased 2-path estimation, unbiased butterfly estimation and privacy budget allocation. Section IV provides a systematic evaluation on our work. Section V draws a conclusion.

II. Background

A. Bipartite Graph and LDP

We define a bipartite graph as $G = (U, V, E)$, where $U = \{U_1, ..., U_m\}$ represents m upper-layer nodes, $V = \{V_1, ..., V_n\}$ represents n lower-layer nodes, E represents the edges existing in G. $a_{ij} = 1$ if and only if edge $(U_i, V_j) \in E$. In our method, data is collected from only one side, assuming U. Let $A_i = \{a_{i1}, ..., a_{in}\}$ represent the neighbor list of U_i, and d_i denote the degree of U_i. The adjacency matrix of the bipartite graph is denoted as $A = \{A_1, ..., A_m\}$, which is not asymmetric. Let d_{pr} represent the known average degree by the server, which can be calculated with a minimal consumption of privacy budget. Let d_{ij} be the size of common neighbors of U_i and U_j, which is also the 2-path count containing U_i and U_j, B_{ij} be the number of butterflies containing U_i and U_j, B be the number of butterflies in G.

For privacy protection in graph data, edge-LDP and relationship-DP are proposed. The former ensures that the message sent by U_i does not reveal whether an arbitrary a_{ij} exists, while the latter guarantees that the server cannot infer the existence of an arbitrary $a_{ij}, 1 \leq i \leq m, 1 \leq j \leq n$ according to all the collected information.

Definition 1. *(Edge local differential privacy [8]). A randomized algorithm R satisfies ε-edge LDP, if for any two adjacency bit vectors a_i and a_i' that differ in one bit, and any output $s \in range(R)$, $\frac{Pr[R(a)=s]}{Pr[R(a_i')=s]} \leq e^{\varepsilon}$ holds.*

Definition 2. *(Relationship differential privacy [9]). For $i \in [n]$, let R_i be a local randomizer of user v_i that takes a_i as input. $(R_1, ..., R_n)$ satisfies ε-relationship DP if for any two adjacency graphs G and G' that differ in one edge, and any $(s_1, ..., s_n) \in Range(R_1) \times ... \times Range(R_n)$, $\frac{Pr[(R_1(a_1),...,R_n(a_n))=(s_1,...,s_n)]}{Pr[(R_1(a_1'),...,R_n(a_n'))=(s_1,...,s_n)]} \leq e^{\varepsilon}$ holds.*

We use $\tilde{X}$ to represent **obfuscated** X, $\hat{X}$ to represent **estimated** X, where X can be replaced with any other symbols. Laplace mechanism [3] and randomized response (RR) [16] are both classic LDP perturbation method. Let $\texttt{Lap}(\frac{1}{\varepsilon})$ denote Laplace noise with mean 0 scale $\frac{1}{\varepsilon}$, $\texttt{RR}_{\varepsilon}(A_i)$ flip each 0/1 bit in A_i with probability $\frac{1}{e^{\varepsilon}+1}$. Due to the nature of bipartite graphs, the edges corresponding to U_i and U_j do not overlap, allowing Laplace mechanism and RR to provide same level of edge-LDP and relationship-DP.

B. Related Work

DP-based graph analysis assumes a trusted third party [17]–[20]. For systematic privacy preservation [21], there have been numerous LDP-based triangle counting works [6], [7], [9] in undirected graphs. A non-interactive triangle counting method is proposed in [9], however, its running time is too long for large-scale graphs. With an additional interaction, accurate and efficient triangle counting is achieved in [9]. However, the communication cost in [9] is too high, so [6] reduces it with a sampling mechanism. Utilizing approximation, a study in [7] achieves efficient non-interactive triangle counting but lacks estimation accuracy. Similarly, researchers propose non-interactive triangle counting to avoid synchronization issues [14] under shuffle DP [15]. Triangle counting works inspire butterfly counting in bipartite graphs.

Similar to triangles in ordinary graphs, butterflies are the smallest closed unit in bipartite graphs. Butterfly counting is fundamental in bipartite graph analysis. Method in [14] counts butterflies in bipartite graphs non-interactively, but do not fully leverage the characteristics of bipartite graphs, leading to insufficient estimation accuracy. A study in [1] introduces multi-round interactions and complex sensitivity analysis to achieve accurate butterfly counting in bipartite graphs, however, multi-round interactions require user synchronization. Both interactive and non-interactive methods are proposed in [2], where the interactive method offers faster computation but incurs more noise and communication cost, while the non-interactive method provides higher accuracy but suffers from high time complexity. Overall, these methods can be used for butterfly counting but fail to balance estimation accuracy, communication cost, running efficiency and user synchronization.

III. Our Method PBC

A. Unbiased 2-path Estimation

We choose the setting where each node can only see its neighbors, similar to [6], [7], [9]. When performing subgraph counting with interactive methods in this setting, the transmission of noisy subgraphs is required [6], [9], and each user must wait for the server's response before continuing to upload data [14], leading to issues of communication cost and user synchronization. Non-interactive subgraph counting methods offer a solution to these problems. Therefore, we design a non-interactive method for butterfly counting in bipartite graphs. There are two existing non-interactive butterfly counting schemes, WLocal [14] and DBE [2]. WLocal uses node sampling to avoid repeated consumption of privacy budgets, leading to insufficient estimation accuracy. DBE involves subgraph matching in dense noisy graphs, resulting in high

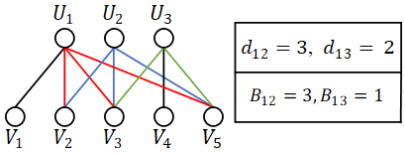


Fig. 2: The numbers of d_{ij} and B_{ij} in graph, and the relationship between d_{ij} and B_{ij}.

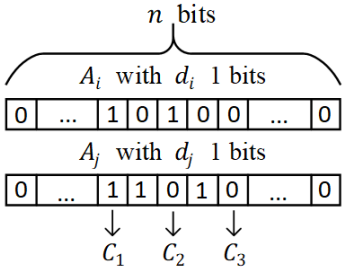


Fig. 3: The neighbor lists of U_i and U_j, each V_i corresponds to one of the C_1, C_2, C_3 three relationships with U_i and U_j.

time complexity. Hence, our designed PBC aims to achieve high estimation accuracy with low time complexity.

As shown in Fig. 2, in bipartite graphs, a butterfly must contain two U nodes and two V nodes, because there is no edge between two nodes from the same layer. Butterflies containing (U_i, U_j) do not overlap with those containing any other $(U_k, U_l), k \neq i$ or $l \neq j$. If we can obtain an unbiased estimate of butterfly counts containing any (U_i, U_j), we can estimate the global number of butterflies in the bipartite graph. Directly analyzing the shape of butterfly containing four nodes would inevitably encounter the same issue as [2], which is subgraph matching in dense noisy graphs. We find that butterfly counts containing (U_i, U_j) is a function of their common neighbor count d_{ij}, denoted as $B_{ij} = C_{d_{ij}}^2$. Therefore, we first derive an unbiased estimate of d_{ij}.

Privacy. In PBC, the server collects noisy degrees and neighbor lists from U nodes with privacy budget ε, achieving $\varepsilon-$relationship DP and $\varepsilon-$edge LDP. The degree is perturbed with Laplace mechanism as $\tilde{d}_i = d_i + \texttt{Lap}(\frac{1}{\varepsilon_0})$, and the neighbor list is perturbed with $\tilde{A}_i = \texttt{RR}_{\varepsilon_1}(A_i)$, where each bit in A_i flips with probability $\frac{1}{e^{\varepsilon_1}+1}$, and $\varepsilon_0 + \varepsilon_1 = \varepsilon$.

The server constructs the noisy graph $\tilde{G}$ based on $\tilde{A} = \{\tilde{A}_1, ..., \tilde{A}_m\}$, does not need to send any information to nodes, so PBC avoids **user synchronization**. We aim to estimate the number of common neighbors d_{ij} between U_i and U_j. Let A_i contain d_i 1 bits, A_j contain d_j 1 bits, we first calculate $\tilde{d}_{ij}$, the number of common neighbors between U_i and U_j in $\tilde{G}$. Let $P = \frac{e^{\varepsilon_1}}{e^{\varepsilon_1}+1}$, we categorize the relationships between V_k and (U_i, U_j) in G into three cases $\{C_1, C_2, C_3\}$, then calculate the expected number of common neighbors between U_i and U_j in $\tilde{G}$ for each case:

- C_1: As shown in Fig. 3, C_1 means V_k is a common neighbor of U_i and U_j. This case occurs d_{ij} times, V_k remains a common neighbor of U_i and U_j in $\tilde{G}$ with a probability of P^2. The expectation is $d_{ij} \times P^2$.
- C_2: As shown in Fig. 3, C_2 means V_k is a neighbor of either U_i or U_j (but not both). This case occurs $d_i + d_j - 2d_{ij}$ times, V_k converts to a common neighbor of U_i and U_j in $\tilde{G}$ with a probability of $P(1-P)$. The expectation is $(d_i + d_j - 2d_{ij}) \times P(1-P)$.
- C_3: As shown in Fig. 3, C_3 means V_k is neither a neighbor of U_i nor U_j. This case occurs $n - d_i - d_j + d_{ij}$ times, V_k converts to a common neighbor of U_i and U_j in $\tilde{G}$ with a probability of $(1-P)^2$. The expectation is $(n - d_i - d_j + d_{ij}) \times (1-P)^2$.

The sum of these three expressions gives the total expected number of $\tilde{d}_{ij}$ as follows:

$$\tilde{d}_{ij} = d_{ij}P^2 + (d_i + d_j - 2d_{ij})P(1-P) + (n - d_i - d_j + d_{ij})(1-P)^2$$

Solving this expression allows us to estimate d_{ij} according to the noisy data as Eq. (1). $\tilde{d}_{ij}$ can be efficiently computed using $M = \tilde{A} \times \tilde{A}^T$, where $\tilde{d}_{ij} = M_{ij}$.

$$\hat{d}_{ij} = \frac{\tilde{d}_{ij} - (\tilde{d}_i + \tilde{d}_j)(2P-1)(1-P) - n(1-P)^2}{(2P-1)^2} \quad (1)$$

Now we can estimate d_{ij} with high **running efficiency**, which serves as the foundation for estimating the number of butterflies B_{ij} containing (U_i, U_j). The approach leverages the characteristics of bipartite graphs, making the estimation of d_{ij} accurate and efficient. In PBC, each user only needs to upload $\tilde{d}_i$ and $\tilde{A}_i$, so the **communication cost** is low as $O(n)$.

B. Unbiased Butterfly Estimation

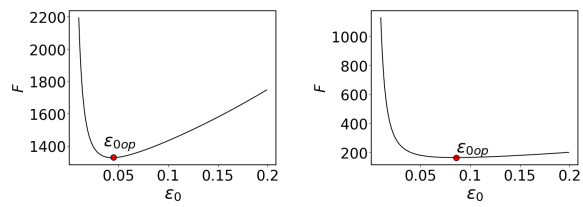
Now we address the issues of running efficiency, communication cost and user synchronization, the next step is to achieve estimation accuracy. Since we obtain an unbiased $\hat{d}_{ij}$, a simple idea for butterfly containing (U_i, U_j) estimation is $\hat{B}_{ij} = C_{\hat{d}_{ij}}^2$. However, as analyzed in [14], [22], this leads to biased estimation due to the noise in $\hat{d}_{ij}$. Let $\hat{d}_{ij} = d_{ij} + \triangle_{ij}$, where $\triangle_{ij}$ is noise with an expected value of 0, because $\hat{d}_{ij}$ is unbiased. The expected butterfly count can be expressed as follows, where $D(\hat{d}_{ij})$ is the variance of $\hat{d}_{ij}$.

$$\begin{aligned}\hat{B}_{ij} &= E(\frac{d_{ij}(d_{ij}-1)}{2}) \\ &= \frac{\hat{d}_{ij}(\hat{d}_{ij}-1)}{2} - \frac{\triangle_{ij}^2}{2} \\ &= \frac{\hat{d}_{ij}(\hat{d}_{ij}-1)}{2} - \frac{1}{2}D(\hat{d}_{ij})\end{aligned}$$

Now we transform the unbiased estimation of B_{ij} into a function of $\hat{d}_{ij}$. Then we need to compute $D(\hat{d}_{ij})$. According to Eq. (1), $\tilde{d}_{ij}$, $\tilde{d}_i$ and $\tilde{d}_j$ are independent of each other, we can derive $D(\hat{d}_{ij})$ as follows.

$$\begin{aligned}&D(\hat{d}_{ij}) \\ =&\frac{1}{(2P-1)^4}D(\tilde{d}_{ij}) + \frac{(1-P)^2}{(2P-1)^2}\frac{4}{\varepsilon_0^2}\end{aligned} \quad (2)$$

Now, we need to calculate the variance of $\tilde{d}_{ij}$. As we analyzed, $\tilde{d}_{ij}$ arises from three cases, each following a binomial



(a) MOOC (b) 100K

Fig. 4: The trends of F across various ε_0 on different datasets when $\varepsilon = 2$.

distribution. Since the three cases are independent of each other, the variance of $\tilde{d}_{ij}$ can be calculated as the sum of the variances of these three cases as Eq. (3).

$$\begin{aligned} & D(\tilde{d}_{ij}) \\ =& d_{ij}P^2(1-P^2)+ \\ & (d_i+d_j-2d_{ij})P(1-P)(1-P(1-P))+ \\ & (n-d_i-d_j+d_{ij})(1-P)^2(1-(1-P)^2) \\ \approx& (\tilde{d}_i+\tilde{d}_j)P(2P-1)(1-P)+nP(1-P)^2(2-P) \end{aligned} \tag{3}$$

Now we can calculate the unbiased estimation of B_{ij} as Eq. (4), where $D(\tilde{d}_{ij})$ can be calculated according to Eq. (3).

$$\hat{B}_{ij} = \frac{\hat{d}_{ij}(\hat{d}_{ij}-1)}{2} - \frac{1}{2(2P-1)^4}D(\tilde{d}_{ij}) - \frac{(1-P)^2}{(2P-1)^2}\frac{2}{\varepsilon_0^2} \tag{4}$$

By converting $\hat{B}_{ij}$ into a function of $\hat{d}_{ij}$, calculating the variance of $\hat{d}_{ij}$, we achieve the **estimation accuracy** for butterfly counting. Now we address the four issues we mentioned.

C. Privacy Budget Allocation

We can unbiasedly estimate the butterfly count according to $\tilde{d} = \{\tilde{d}_1, ..., \tilde{d}_m\}$ and $\tilde{A} = \{\tilde{A}_1, ..., \tilde{A}_m\}$, which involves two rounds of data collection. The allocation of privacy budget between two rounds impacts the estimation accuracy [7], so we aim to design an automated privacy budget allocation method to enhance data utility. Similar to [7], we assume that the server knows the average degree d_{pr}, which can be computed with minimal privacy budget consumption. Since $\hat{B}_{ij}$ is a function of $\hat{d}_{ij}$, we want to allocate the privacy budget to minimize $D(\hat{d}_{ij})$. Assuming any $d_i = d_{pr}$, and all the 1 bits are uniformly randomly distributed, we can derive as follows.

$$\begin{aligned} & D(\hat{d}_{ij}) \\ =& \frac{2d_{pr}P(1-P)}{(2P-1)^3} + \frac{nP(1-P)^2(2-P)}{(2P-1)^4} + \frac{(1-P)^2}{(2P-1)^2}\frac{4}{\varepsilon_0^2} \end{aligned}$$

Now we can express $D(\hat{d}_{ij})$ as a function of $d_{pr}, n, \varepsilon_0, \varepsilon_1$. Thus, we can formulate $D(\hat{d}_{ij})$ as Eq. (5), where $\varepsilon_0 + \varepsilon_1 = \varepsilon$. As shown in Fig. 4, when ε is fixed, $D(\hat{d}_{ij})$ first decreases and then increases as ε_0 increases. We select $(\varepsilon_0, \varepsilon_1)$ that minimizes Eq. (5) as the allocated privacy budget.

$$D(\hat{d}_{ij}) = F(d_{pr}, n, \varepsilon_0, \varepsilon_1) \tag{5}$$

Algorithm 1: Framework of PBC

Input : Degrees $\{d_1, ..., d_m\}$, neighbour lists $\{A_1, ..., A_m\}$, privacy budget ε, prior knowledge d_{pr} and the number of V nodes n
Output: The estimated number of butterflies $\hat{B}$

Server:
Calculates $(\varepsilon_0, \varepsilon_1)$ that minimizes $F(d_{pr}, n, \varepsilon_0, \varepsilon_1)$
Nodes:
foreach U_i **do**
  Downloads $(\varepsilon_0, \varepsilon_1)$ from Server;
  Calculates $\tilde{d}_i = d_i + \texttt{Lap}(\frac{1}{\varepsilon_0})$ and $\tilde{A}_i = \texttt{RR}_{\varepsilon_1}(A_i)$;
  Uploads $\tilde{d}_i$, $\tilde{A}_i$ to the server;
end
Server executes:
Constructs the noisy bipartite adjacency matrix $\tilde{A} = \{\tilde{A}_1, ..., \tilde{A}_m\}$;
Calculates the number of noisy common neighbors between any two U nodes $M = \tilde{A} \times \tilde{A}^T$;
Calculates unbiased estimation of 2-paths involving any two U nodes $\hat{d}_{ij}$ according to Eq. (1), with $\tilde{d}_{ij} = M_{ij}$;
Calculates unbiased estimation of butterflies involving any two U nodes $\hat{B}_{ij}$ according to Eq. (4);
$\hat{B} = \sum_{i=1}^{m-1}\sum_{j=i+1}^{m}\hat{B}_{ij}$

We implement PBC in section III, the overall workflow of PBC is as shown in Algorithm 1.

IV. Experiments

A. Experimental Set-up

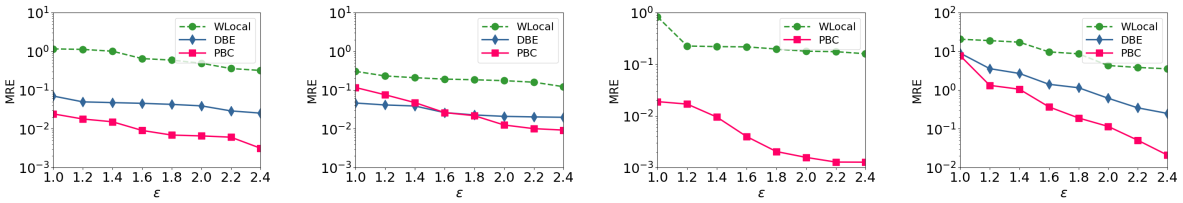
In this section, we construct systematic experiments to verify the effectiveness of PBC. We aim to answer the following three questions:

Q1. Does PBC accurately estimate the number of butterflies?
Q2. Does PBC efficiently estimate the number of butterflies?
Q3. Does PBC's privacy budget allocation enhance data utility?

For Q1, we compare mean relative error (MRE) on butterfly counting of PBC with WLocal [14] and DBE [2] across different datasets and privacy budgets. WLocal is the state-of-the-art efficient non-interactive butterfly counting scheme. **For Q2**, we compare the running time of PBC and DBE [2] across different privacy budgets. DBE is the state-of-the-art accurate non-interactive butterfly counting scheme. **For Q3**, we compare MREs of PBC with different privacy budget allocation parameters across different datasets and ε.

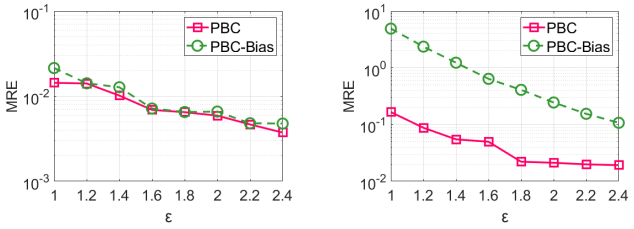
Datasets. Our experiments are conducted on four open-source real-world datasets, which can be accessed via Stanford and GroupLens.

1) MOOC. Student actions on a MOOC platform, edges represent the actions by users on the targets, with 97 U nodes, 7047 V nodes and 178443 edges.



(a) MOOC (b) 100K (c) 10M (d) Wiki

Fig. 5: Comparison on MREs of butterfly counting between WLocal, DBE and PBC across various ε on different datasets.



(a) MOOC (b) 100K

Fig. 6: Comparison on MREs of butterfly counting between PBC and PBC-Bias across various ε on different datasets.

2) 100K. 100K movie ratings, edges represent the users rating movies, with 943 U nodes, 1682 V nodes and 100000 edges.
3) 10M. 10M movie ratings, edges represent the users rating movies, with 10677 U nodes, 69878 V nodes and 10000000 edges.
4) Wiki. Wikipedia adminship election data, edges represent the users voting for the editors, with 2391 U nodes, 6210 V nodes and 110087 edges.

All the experiments are conducted using Python 3.7 on a desktop computer equipped with an Intel(R) Xeon(R) Gold 6242 CPU and 128GB of RAM, running on a Linux system.

B. Evaluation on Accuracy of PBC

In section IV-B, we answer Q1 to demonstrate the accuracy of PBC. To answer this, we evaluate PBC and the necessity of obtaining unbiased butterfly counts with variance analysis. First, we compare MREs of PBC and WLocal across different datasets, with ε as the independent variable. As shown in Fig. 5, PBC significantly outperforms WLocal in all datasets under all ε, with the improvement becoming more pronounced as ε increases. Besides, except for the case where $\varepsilon < 1.6$ on the 100K dataset, PBC consistently outperforms DBE, and the improvement becomes more pronounced as ε increases. We attribute this to the smaller variance introduced by the Laplace mechanism. The 10M dataset is too large, causing DBE's running time to be excessively long, so DBE's results on 10M are not included in Fig. 5(c). We attribute PBC's high accuracy to avoiding node sampling and employing a refined calibration mechanism.

Next, we experimentally validate the effectiveness of using variance analysis to compute unbiased butterflies. We refer to the method without variance analysis as PBC-Bias and compare MREs of PBC and PBC-Bias across different datasets. As shown in Fig. 6, PBC consistently achieves similar or smaller MREs compared to PBC-Bias across all datasets and ε, with the improvement becoming more pronounced as ε decreases. Because as ε decreases, the variance component becomes larger, which results in PBC-Bias having a larger bias error. On MOOC, the optimization of PBC compared to PBC-Bias is small because B is large and m is small on MOOC. The bias error introduced by PBC-Bias is comparatively small in relation to the total number of butterflies.

We also compare the MREs claimed by DBE [2] and BC-LimBN [1] on specific datasets with PBC. The accuracy of PBC achieves similar results to DBE, when $\varepsilon = 2$ on 10M. In comparison to BC-LimBN, PBC achieves better accuracy on both 10M and Wiki for any ε. We do not include BC-LimBN as a competitor due to its complex interaction design, and we are unable to reproduce the claimed accuracy. Now we answer Q1, demonstrating the estimation accuracy of PBC. We believe that DBE outperforms PBC only when ε is small, the graph is dense, and the graph scale is small.

C. Evaluation on Efficiency and Allocation of PBC

ε	1	1.5	2	2.5	3
DBE	9273	1079	582	4644	2220
PBC	145	145	144	144	144
WLocal	21	21	20	19	19

TABLE I: Comparison on running time (s) between DBE, PBC and WLocal across various ε on 10M.

In section IV-C, we answer Q2 and Q3 to demonstrate the running efficiency of PBC and efficacy of privacy budget allocation. We first compare the running times of DBE, PBC, and WLocal under different ε on 10M. As shown in Table. I, the running times of the three methods decrease sequentially, with PBC achieving two orders of magnitude improvement in running efficiency compared to DBE. It is worth noting that the running time of PBC does not change significantly with varying ε, as its time complexity mainly stems from matrix multiplication. The higher running efficiency of WLocal compared to PBC is mainly due to the node sampling

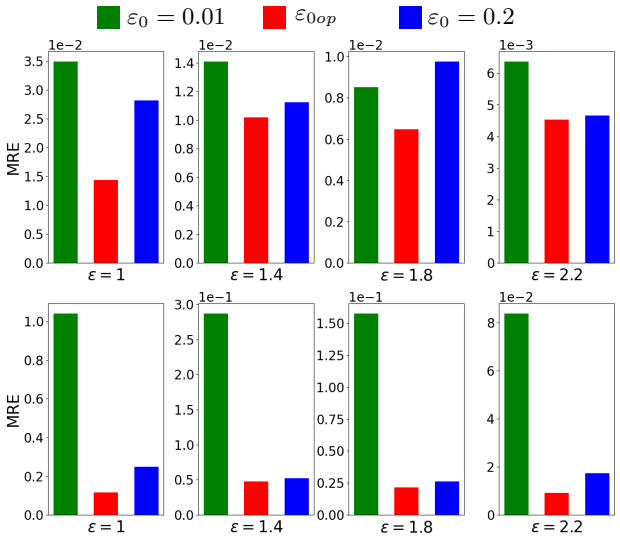


Fig. 7: Comparison on MREs between PBC with allocated ε_{0op}, $\varepsilon_0 = 0.01$ and $\varepsilon_0 = 0.2$ across various ε on MOOC and 100K, the first row is results on MOOC, the second row is results on 100K.

operation, which also leads to insufficient accuracy in butterfly counting. Considering both estimation accuracy and running efficiency, we conclude that PBC achieves a good balance, achieving similar estimation accuracy to DBE with two orders of magnitude lower running time, while achieving significantly higher estimation accuracy than WLocal with an acceptable running time. Now we answer Q2, demonstrating the high running efficiency of PBC.

ε	1	1.4	1.8	2.2
MOOC	0.04	0.05	0.05	0.05
100K	0.07	0.08	0.09	0.1

TABLE II: Values of allocated privacy budget ε_{0op} across various ε on MOOC and 100K.

Now we discuss the efficacy of the privacy budget allocation in PBC to answer Q3. As shown in Table. II, ε_{0op} for MOOC and 100K ranges from 0.01 to 0.2 under different ε. Therefore, we compare the performance of $\varepsilon_0 = 0.01$, $\varepsilon_0 = \varepsilon_{0op}$ and $\varepsilon_0 = 0.2$ across different ε on MOOC and 100K. From Fig. 7, it can be seen that ε_{0op} consistently achieves comparable or better results than the other two parameters. Specifically, ε_{0op} achieves one order of MRE deduction compared to $\varepsilon_0 = 0.01$ on 100K, demonstrating that arbitrarily selecting ε_0 is not advisable. ε_{0op} varies across different datasets and ε, therefore, an automated privacy budget allocation method helps enhance data utility. Now we answer Q3, demonstrating the efficacy of privacy allocation method in PBC.

V. Conclusion

This paper presents PBC, a novel non-interactive butterfly counting method in bipartite graphs, achieving high estimation accuracy and running efficiency, avoiding high communication cost and user synchronization. By converting the butterfly count into a function of the 2-path count, PBC achieves a two-order-of-magnitude improvement in running efficiency, while obtaining an unbiased butterfly counts without sampling, effectively enhancing estimation accuracy. The privacy budget allocation method in PBC also enhances data utility. Our work does not involve interactions between users and the server, which reduces communication cost and user synchronization issues, and PBC can be easily implemented under shuffle DP model to further improve estimation accuracy.

References

[1] M. Wang, H. Jiang, P. Peng, Y. Li, and W. Huang, "Toward accurate butterfly counting with edge privacy preserving in bipartite networks," in *Proc. of IEEE INFOCOM*, 2024.
[2] Y. He, K. Wang, W. Zhang, X. Lin, W. Ni, and Y. Zhang, "Butterfly counting over bipartite graphs with local differential privacy," in *Proc. of IEEE ICDE*, 2024.
[3] C. Dwork, "Differential privacy," in *Automata, Languages and Programming*, 2006.
[4] J. C. Duchi, M. I. Jordan, and M. J. Wainwright, "Local privacy and statistical minimax rates," in *Proc. of IEEE FOCS*, 2013.
[5] S. P. Kasiviswanathan, H. K. Lee, K. Nissim, S. Raskhodnikova, and A. Smith, "What can we learn privately?" *SICOMP*, vol. 40, 2011.
[6] J. Imola, T. Murakami, and K. Chaudhuri, "Communication-Efficient triangle counting under local differential privacy," in *Proc. of USENIX Security*, 2022.
[7] Q. Ye, H. Hu, M. H. Au, X. Meng, and X. Xiao, "Lf-gdpr: A framework for estimating graph metrics with local differential privacy," *TKDE*, vol. 34, 2022.
[8] Z. Qin, T. Yu, Y. Yang, I. Khalil, X. Xiao, and K. Ren, "Generating synthetic decentralized social graphs with local differential privacy," in *Proc. of ACM CCS*, 2017.
[9] J. Imola, T. Murakami, and K. Chaudhuri, "Locally differentially private analysis of graph statistics," in *Proc. of USENIX Security*, 2021.
[10] L. Hou, W. Ni, S. Zhang, N. Fu, and D. Zhang, "Wdt-scan: Clustering decentralized social graphs with local differential privacy," *Computers & Security*, vol. 125, 2023.
[11] ——, "Ppdu: dynamic graph publication with local differential privacy," *KAIS*, vol. 65, 2023.
[12] L. Jiang, Y. Yan, Z. Tian, Z. Xiong, and Q. Han, "Personalized sampling graph collection with local differential privacy for link prediction," *WWW*, vol. 26, 2023.
[13] T. Guo, S. Peng, Y. Li, M. Zhou, and T.-K. Truong, "Community-based social recommendation under local differential privacy protection," *Information Sciences*, vol. 639, 2023.
[14] J. Imola, T. Murakami, and K. Chaudhuri, "Differentially private triangle and 4-cycle counting in the shuffle model," in *Proc. of ACM CCS*, 2022.
[15] U. Erlingsson, V. Feldman, I. Mironov, A. Raghunathan, K. Talwar, and A. Thakurta, "Amplification by shuffling: from local to central differential privacy via anonymity," in *Proc. of SIAM SODA*, 2019.
[16] S. L. Warner, "Randomized response: A survey technique for eliminating evasive answer bias," *JASA*, vol. 60, 1965.
[17] M. Hay, C. Li, G. Miklau, and D. Jensen, "Accurate estimation of the degree distribution of private networks," in *2009 Ninth IEEE International Conference on Data Mining*, 2009.
[18] V. Karwa, S. Raskhodnikova, A. Smith, and G. Yaroslavtsev, "Private analysis of graph structure," *Proc. VLDB Endow.*, vol. 4, 2011.
[19] J. Zhang, G. Cormode, C. M. Procopiuc, D. Srivastava, and X. Xiao, "Private release of graph statistics using ladder functions," in *Proc. of ACM SIGMOD*, 2015.
[20] X. Jian, Y. Wang, and L. Chen, "Publishing graphs under node differential privacy," *TKDE*, vol. 35, 2023.
[21] F. Li, H. Li, B. Niu, and J. Chen, "Privacy computing: Concept, computing framework, and future development trends," *Engineering*, vol. 5, 2019.
[22] Q. Hillebrand, V. Suppakitpaisarn, and T. Shibuya, "Unbiased locally private estimator for polynomials of laplacian variables," in *Proc. of ACM SIGKDD*, 2023.